Eloquent Relationships: Deep Dive — Study Notes

Course: Laracasts — Eloquent Relationships: Deep Dive (Jeremy McPeak, 8 episodes, ~2h 11m) **Stack target:** Laravel 11/12, PHP 8.2+ **Domain:** User, Profile, Task, Team, Department, Description, Note, Tag

---

Recurring Themes (read this first)

- Every relationship has **two layers**: the migration (DB) + the model methods (Eloquent). Wire both.
- **Property** = read data. **Method** = manipulate / query.
- Always eager-load. N+1 is the default failure mode.
- `withCount` / `withSum` aggregate in SQL — never aggregate in PHP.
- Polymorphic = one column linking to many model types via `*_type`.

Episode 1 — One-to-One

1. Concept

`User` has one `Profile`. Profile holds the FK.

2. Migration

```
php artisan make:model Profile -m
```

```
Schema::create('profiles', function (Blueprint $table) {
    $table->id();
    $table->string('handle')->nullable();
    $table->text('bio')->nullable();
    $table->foreignId('user_id')
          ->constrained()
          ->cascadeOnDelete()
          ->unique(); // ← this is what makes it 1:1
    $table->timestamps();
});
```

`constrained()` = referential integrity. `unique()` = enforces "one." Without it you've got 1:many in disguise.

3. Models

```
// User.php
public function profile()
{
    return $this->hasOne(Profile::class);
}
```

```
// Profile.php
protected $fillable = ['handle', 'bio'];

public function user()
{
    return $this->belongsTo(User::class);
}
```

4. Override the FK convention

```
return $this->hasOne(Profile::class, 'owner_id', 'id');
return $this->belongsTo(User::class, 'owner_id', 'id');
```

5. Property vs Method

```
$user->profile;            // fetch (Profile instance)
$user->profile->handle;

$user->profile();          // builder — chainable
$user->profile()->where('handle', 'like', 'j%')->first();
$user->profile()->create([...]);
```

6. `withDefault()` — kill null checks

```
public function profile()
{
    return $this->hasOne(Profile::class)->withDefault([
        'handle' => 'no profile set',
        'bio'    => 'no profile set',
    ]);
}
```

7. Two ways to create

```
$user->profile()->create(['handle' => 'jeremy', 'bio' => '...']);
$user->profile()->save(new Profile(['handle' => 'jeremy']));
```

8. The N+1 problem

```
// 1 + 10 queries
$users = User::all();
foreach ($users as $u) echo $u->profile->handle;

// 2 queries
$users = User::with('profile')->get();
```

9. Inverse

```
Profile::find(1)->user;
```

Cheat Sheet

```
$this->hasOne(Profile::class);                  // parent → child
$this->belongsTo(User::class);                  // child → parent
$this->hasOne(...)->withDefault([...]);         // null-safe
User::with('profile')->get();                   // eager
```

Common Mistakes

- Forgetting `unique()` on the FK.
- Defining `hasOne` but not `belongsTo` (or vice versa).
- `$user->profile()` when you meant `$user->profile`.
- Skipping `with()`.

Episode 2 — One-to-Many

1. Concept

`User` has many `Tasks`. Same shape as 1:1, drop the `unique()`.

2. Migration

```
Schema::create('tasks', function (Blueprint $table) {
    $table->id();
    $table->string('title');
    $table->string('status'); // 'C', 'H', 'A'
    $table->foreignId('user_id')->constrained();
    $table->timestamps();
});
```

No `cascadeOnDelete()` here — orphan tasks may be desirable (reassign instead of nuke).

3. Models

```
// User.php
public function tasks() { return $this->hasMany(Task::class); }
```

```
// Task.php
protected $fillable = ['title', 'status'];

public function user() { return $this->belongsTo(User::class); }
```

4. Factory

```
public function definition(): array
{
    return [
        'title'  => fake()->sentence(),
        'status' => fake()->randomElement(['C', 'H', 'A']),
    ];
}
```

Don't set `user_id` here — let the seeder assign via the relationship.

5. Seeder

```
User::factory()
    ->count(10)
    ->has(Profile::factory())
    ->has(Task::factory()->count(10))
    ->create();
```

```
php artisan migrate:fresh --seed
```

6. Eager loading multiple

```
User::with(['profile', 'tasks'])->get();
```

7. Constrained eager loading

```
User::with([
    'tasks' => fn ($q) => $q->where('status', 'A'),
])->find(1);
```

8. Filter via the method

```
$user->tasks()->where('status', 'A')->get();
$user->tasks()->where('status', 'C')->count();
```

Cheat Sheet

```
$this->hasMany(Task::class);
User::with(['tasks' => fn ($q) => $q->where(...)])->get();
$user->tasks()->where(...)->get();
```

Common Mistakes

- Setting `user_id` in the factory `definition()` and double-assigning.
- Calling `$user->tasks->where(...)` (collection) when you wanted `$user->tasks()->where(...)` (builder).
- Forgetting `$fillable` on Task.

Episode 3 — Many-to-Many

1. Concept

`User` ↔ `Team`. Pivot table required. Convention: alphabetical, singular, snake_case → `team_user`.

2. Pivot migration

```
php artisan make:migration create_team_user_table
```

```
Schema::create('team_user', function (Blueprint $table) {
    $table->foreignId('user_id')->constrained()->cascadeOnDelete();
    $table->foreignId('team_id')->constrained()->cascadeOnDelete();
    $table->string('role')->default('member');
    $table->primary(['user_id', 'team_id']);   // composite PK, no `id`
    $table->timestamps();
});
```

3. Models (both sides)

```
// User.php
public function teams()
{
    return $this->belongsToMany(Team::class)
                ->withPivot('role')
                ->withTimestamps();
}

// Team.php
public function users()
{
    return $this->belongsToMany(User::class)
                ->withPivot('role')
                ->withTimestamps();
}
```

`withPivot('role')` is required to expose the column. Without it, `pivot->role` is null.

4. Seeding with pivot data

```
$user->teams()->attach($team->id, [
    'role' => collect(['owner', 'member', 'guest'])->random(),
]);
```

5. Hide pivot from JSON

```
$user->teams->makeHidden('pivot');

// or globally on Team:
protected $hidden = ['pivot'];
```

6. Read pivot data

```
$user->teams->first()->pivot->role;
$user->teams->first()->pivot->created_at;
```

7. Custom Pivot model

```
// Membership.php
use Illuminate\Database\Eloquent\Relations\Pivot;

class Membership extends Pivot
{
    protected $casts = ['created_at' => 'datetime'];

    public function getIsOwnerAttribute(): bool
    {
        return $this->role === 'owner';
    }
}
```

Hook up on **both** ends:

```
return $this->belongsToMany(Team::class)
            ->using(Membership::class)
            ->withPivot('role')
            ->withTimestamps();
```

```
$user->teams->first()->pivot->is_owner; // true/false
```

8. Counting: accessor vs `withCount`

```
// Accessor — runs a query every access (N+1 trap)
public function getTeamsCountAttribute(): int
{
    return $this->teams()->count();
}

// withCount — single query, eager
User::withCount('teams')->get()->first()->teams_count;
```

9. Sync (replace pivot set)

```
$team->users()->sync([
    1 => ['role' => 'owner'],
    3 => ['role' => 'guest'],
]);
```

After this, only users 1 and 3 are on the team.

10. Validation for a sync UI

```
$data = $request->validate([
    'members'         => ['array'],
    'members.*.id'    => ['required', 'integer', 'exists:users,id'],
    'members.*.role'  => ['required', 'in:owner,member,guest'],
]);

$payload = collect($data['members'])
    ->mapWithKeys(fn ($m) => [$m['id'] => ['role' => $m['role']]])
    ->toArray();

$team->users()->sync($payload);
```

11. attach / sync / syncWithoutDetaching / toggle

```
$team->users()->attach($id, [...]);          // add
$team->users()->detach($id);                 // remove (all if no arg)
$team->users()->sync([1, 2, 3]);             // replace — removes missing
$team->users()->syncWithoutDetaching([...]); // add only, no removal
$team->users()->toggle([1, 2]);              // add if absent, remove if present
$team->users()->updateExistingPivot($id, [...]); // pivot cols only
```

Cheat Sheet

```
$this->belongsToMany(Team::class)->withPivot('role')->withTimestamps();
$this->belongsToMany(...)->using(Membership::class);
$user->teams()->attach($id, ['role' => 'owner']);
$team->users()->sync([1 => ['role' => 'owner']]);
User::withCount('teams')->get();
```

Common Mistakes

- Forgetting `withPivot('role')` → pivot column reads null.
- Wrong pivot table name without overriding the second arg.
- `attach` in a sync UI → duplicates.
- `sync([1, 2])` (just IDs) when pivot data was needed → defaults applied.
- `using()` on only one side.

Episode 4 — Has Many Through

1. Concept

`Department → User → Task`. Department has many tasks **through** users.

2. Migrations

```
php artisan make:migration add_department_id_to_users_table --table=users
```

```
Schema::table('users', function (Blueprint $table) {
    $table->foreignId('department_id')
          ->nullable()
          ->constrained()
          ->nullOnDelete();
});
```

`nullOnDelete()` = delete department, keep users, null their FK.

```
// departments
Schema::create('departments', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->timestamps();
});
```

3. Models

```
// User.php
public function department() { return $this->belongsTo(Department::class); }
```

```
// Department.php
protected $fillable = ['name'];

public function users() { return $this->hasMany(User::class); }

public function tasks()
{
    return $this->hasManyThrough(Task::class, User::class);
    //                           ↑ final     ↑ intermediate
}
```

4. Factory + seeder

```
// DepartmentFactory.php
return ['name' => fake()->company()];
```

```
$departments = Department::factory()->count(3)->create();

User::factory()
    ->count(10)
    ->state(fn () => ['department_id' => $departments->random()->id])
    ->has(Profile::factory())
    ->has(Task::factory()->count(10))
    ->create();
```

5. Route + view

```
Route::get('/departments', function () {
    return view('departments.index', [
        'departments' => Department::with(['users', 'tasks'])->get(),
    ]);
});
```

```
@foreach ($departments as $department)
    <h2>{{ $department->name }}</h2>
    <p>{{ $department->users->count() }} people, {{ $department->tasks->count() }} tasks</p>
    @foreach ($department->tasks as $task)
        <li>{{ $task->title }} — {{ $task->status }}</li>
    @endforeach
@endforeach
```

6. Querying through

```
$department->tasks()->where('status', 'C')->get();
```

One SQL with inner joins — vs writing nested loops yourself.

Cheat Sheet

```
$this->hasManyThrough(Task::class, User::class);
$department->tasks()->where(...)->get();
```

Common Mistakes

- Wrong arg order — it's `(Related, Through)`.
- Reaching for it when the middle layer is a pivot, not an entity (use `belongsToMany`).
- Expecting pivot-style data — there isn't any.

Episode 5 — Polymorphic (`morphOne` / `morphMany`)

1. Concept

One table relates to many model types via `*_id` + `*_type`.

2. Description migration

```
Schema::create('descriptions', function (Blueprint $table) {
    $table->id();
    $table->string('content');
    $table->morphs('describable'); // describable_id + describable_type, indexed
    $table->timestamps();
});
```

3. Description model

```
protected $fillable = ['content'];

public function describable()
{
    return $this->morphTo();
}
```

4. Parents (User, Team)

```
public function description()
{
    return $this->morphOne(Description::class, 'describable');
}
```

5. Seeding

```
$user->description()->create(['content' => 'Senior Developer']);
$team->description()->create(['content' => 'We build the backend systems.']);
```

6. What's stored

```
SELECT id, content, describable_id, describable_type FROM descriptions;
-- 1 | Senior Developer | 1 | App\Models\User
-- 2 | We build...      | 1 | App\Models\Team
```

7. Morph map (DO THIS)

```
// AppServiceProvider.php boot()
use Illuminate\Database\Eloquent\Relations\Relation;

Relation::enforceMorphMap([
    'users' => \App\Models\User::class,
    'teams' => \App\Models\Team::class,
    'tasks' => \App\Models\Task::class,
]);
```

Now `describable_type` is `'users'` instead of `'App\Models\User'`. Refactor-safe.

8. Querying

```
$user = User::with('description')->first();
$user->description->content;

// Inverse
$description = Description::find(1);
$description->describable; // User or Team instance
```

9. `morphMany` — Note

```
Schema::create('notes', function (Blueprint $table) {
    $table->id();
    $table->string('content');
    $table->morphs('notable');
    $table->timestamps();
});
```

```
// Note.php
protected $fillable = ['content'];
public function notable() { return $this->morphTo(); }
```

```
// User, Team, Task all get this:
public function notes()
{
    return $this->morphMany(Note::class, 'notable');
}
```

10. Seeding multiple

```
$user->notes()->createMany([
    ['content' => 'Reviewed PR #142.'],
    ['content' => 'Pair-programming Tuesday.'],
    ['content' => 'OOO Friday.'],
]);
```

11. Querying

```
User::with('notes')->first()->notes;
```

Cheat Sheet

```
$table->morphs('describable');                          // migration
$this->morphTo();                                       // child
$this->morphOne(Description::class, 'describable');     // one parent
$this->morphMany(Note::class, 'notable');               // many parents
Relation::enforceMorphMap([...]);                       // refactor-safe
```

Common Mistakes

- Typo mismatch between `morphs('describable')` and `morphOne(..., 'descripable')`.
- Skipping the morph map → namespace refactor breaks polymorphic relations silently.
- Forgetting `$fillable` on the polymorphic child.

Episode 6 — Many-to-Many Polymorphic

1. Concept

A single `Tag` attaches to `User`, `Team`, or `Task`. One pivot — `taggables` — with `(tag_id, taggable_id, taggable_type)`.

2. Tag

```
Schema::create('tags', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->timestamps();
});

// Tag.php
protected $fillable = ['name'];
```

3. Pivot migration

```
Schema::create('taggables', function (Blueprint $table) {
    $table->foreignId('tag_id')->constrained()->cascadeOnDelete();
    $table->morphs('taggable'); // taggable_id + taggable_type
});
```

4. On the taggable side (User, Team, Task)

```
public function tags()
{
    return $this->morphToMany(Tag::class, 'taggable');
}
```

5. On the Tag side — `morphedByMany`, NOT `morphToMany`

```
// Tag.php
public function users() { return $this->morphedByMany(User::class, 'taggable'); }
public function tasks() { return $this->morphedByMany(Task::class, 'taggable'); }
public function teams() { return $this->morphedByMany(Team::class, 'taggable'); }
```

Mnemonic: the polymorphic side uses `morphToMany`; the concrete-types side uses `morphedByMany` (Tag is "morphed BY many" different types).

6. Seeding

```
$contractor = Tag::create(['name' => 'contractor']);
$urgent     = Tag::create(['name' => 'urgent']);

$user->tags()->attach($contractor->id);
$team->tags()->attach($contractor->id);
$task->tags()->attach($urgent->id);
```

7. Querying both directions

```
$user->tags;              // Collection<Tag>
$contractor->users;       // Collection<User>
$contractor->teams;       // Collection<Team>

$user->tags->first()->pivot->taggable_type; // 'users' (with morph map)
```

8. Extra pivot columns

```
// migration
$table->timestamp('assigned_at')->nullable();

// model
public function tags()
{
    return $this->morphToMany(Tag::class, 'taggable')->withPivot('assigned_at');
}
```

Cheat Sheet

```
$this->morphToMany(Tag::class, 'taggable');         // taggable side
$this->morphedByMany(User::class, 'taggable');      // tag side
$user->tags()->attach($tag->id);
```

Common Mistakes

- `morphToMany` on Tag instead of `morphedByMany` → returns nothing.
- Pivot named anything other than `{name}s` without overriding.
- No morph map → DB pollution after a namespace move.

Episode 7 — Eager Loading & Aggregates

1. Recap

```
$users = User::with('tasks')->get();          // 2 queries
```

2. Multiple + constrained

```
User::with([
    'tasks' => fn ($q) => $q->where('status', 'C'),
    'teams',
])->first();
```

3. `loadMissing` vs `load`

```
$user = User::with('tasks')->first();

$user->loadMissing(['tasks', 'teams']);  // only loads 'teams' — tasks already loaded
$user->load('tasks');                    // re-runs the query unconditionally
```

Method	Phase	Re-runs if loaded?
`with`	At query	n/a
`load`	Post-fetch	Yes
`loadMissing`	Post-fetch	No

4. `withCount`

```
User::withCount('tasks')->get()->first()->tasks_count;

User::withCount([
    'tasks',
    'tasks as completed_tasks_count' => fn ($q) => $q->where('status', 'C'),
])->get();
```

Attribute name: `{relation}_count` (or your alias).

5. Add `duration` to tasks

```
php artisan make:migration add_duration_to_tasks_table --table=tasks
```

```
Schema::table('tasks', function (Blueprint $table) {
    $table->unsignedInteger('duration')->default(0);
});
```

```
// TaskFactory.php
'duration' => fake()->numberBetween(1, 100),
```

6. Aggregates

```
User::withSum('tasks', 'duration')->first()->tasks_sum_duration;
User::withAvg('tasks', 'duration')->first()->tasks_avg_duration;
User::withMin('tasks', 'duration')->first()->tasks_min_duration;
User::withMax('tasks', 'duration')->first()->tasks_max_duration;

// Constrained
User::withSum([
    'tasks' => fn ($q) => $q->where('status', 'A'),
], 'duration')->first()->tasks_sum_duration;
```

Attribute name: `{relation}_{aggregate}_{column}`.

7. Why aggregates beat PHP-side sums

```
// BAD — pulls every task into memory
$total = $user->tasks->sum('duration');

// GOOD — SUM() in SQL, returns a scalar
User::withSum('tasks', 'duration')->find(1)->tasks_sum_duration;
```

Cheat Sheet

```
User::withCount('tasks');                    // tasks_count
User::withSum('tasks', 'duration');          // tasks_sum_duration
User::withAvg('tasks', 'duration');          // tasks_avg_duration
User::withMin('tasks', 'duration');          // tasks_min_duration
User::withMax('tasks', 'duration');          // tasks_max_duration
$user->loadMissing(['tasks', 'teams']);
```

Common Mistakes

- `withSum('tasks')` without a column → fatal error.
- Treating `tasks_count` as a collection.
- Confusing `with` (query phase) with `load` (post-fetch).
- `$user->tasks->sum('duration')` in a loop.

Episode 8 — Smarter Eager Loading

1. The hidden N+1

```
@foreach ($users as $user)
    <span>{{ $user->name }}</span>
    @foreach ($user->teams as $team) {{ $team->name }} @endforeach
    {{-- forgot to with('teams') → N+1 --}}
@endforeach
```

2. Strict mode — disable lazy loading

```
// AppServiceProvider.php boot()
use Illuminate\Database\Eloquent\Model;

Model::preventLazyLoading(! app()->isProduction());
```

Throws `LazyLoadingViolationException` on any non-eager-loaded access. Crashes loud in dev, graceful in prod.

3. Fix

```
$users = User::with(['profile', 'teams'])->get();

// or
$users = User::with('profile')->get();
$users->loadMissing('teams');
```

4. Auto-loading (the killer feature, Laravel 11+)

```
// Per-collection
$users = User::withCount('teams')->get()->withRelationshipAutoloading();

// Globally — AppServiceProvider.php boot()
Model::automaticallyEagerLoadRelationships();
```

Eloquent watches what your code accesses and batch-loads on first access. Lazy syntax, eager performance.

5. Telescope verifies

```
SELECT * FROM users;
SELECT count(*) FROM team_user WHERE user_id IN (...) GROUP BY user_id;
SELECT * FROM teams INNER JOIN team_user ON ... WHERE user_id IN (...);
```

3 queries regardless of user count. Remove the loop in Blade → the third query disappears entirely.

6. preventLazyLoading vs automaticallyEagerLoadRelationships

Pick one:

- `preventLazyLoading` — strict. Forces explicit `with()`. Good for query-discipline teams.
- `automaticallyEagerLoadRelationships` — pragmatic. Auto-fixes N+1. Good for shipping speed.

Cheat Sheet

```
Model::preventLazyLoading(! app()->isProduction());      // strict
Model::automaticallyEagerLoadRelationships();            // auto
$collection->withRelationshipAutoloading();              // scoped
$user->loadMissing(['rel']);                             // top-up
```

Common Mistakes

- Lazy loading shipped to prod unnoticed.
- `preventLazyLoading()` without the `isProduction()` guard → prod crash on a missed `with()`.
- Mixing `preventLazyLoading` and `automaticallyEagerLoadRelationships`.
- Expecting auto-loading to work without ever accessing the relation — it's lazy detection, not magic.

Master Cheat Sheet

```
// ─── 1:1 ──────────────────────────────────────────────────────────
$table->foreignId('user_id')->constrained()->cascadeOnDelete()->unique();
$this->hasOne(Profile::class);
$this->belongsTo(User::class);

// ─── 1:many ───────────────────────────────────────────────────────
$table->foreignId('user_id')->constrained();
$this->hasMany(Task::class);
$this->belongsTo(User::class);

// ─── many:many ────────────────────────────────────────────────────
// pivot: team_user
$table->foreignId('user_id')->constrained()->cascadeOnDelete();
$table->foreignId('team_id')->constrained()->cascadeOnDelete();
$table->string('role')->default('member');
$table->primary(['user_id', 'team_id']);
$table->timestamps();

$this->belongsToMany(Team::class)->withPivot('role')->withTimestamps();
$user->teams()->attach($id, ['role' => 'owner']);
$team->users()->sync([1 => ['role' => 'owner']]);

// ─── has many through ─────────────────────────────────────────────
$table->foreignId('department_id')->nullable()->constrained()->nullOnDelete();
$this->hasManyThrough(Task::class, User::class);

// ─── polymorphic 1:many ───────────────────────────────────────────
$table->morphs('notable');
$this->morphTo();                                   // child
$this->morphMany(Note::class, 'notable');           // parents

// ─── polymorphic many:many ────────────────────────────────────────
$table->foreignId('tag_id')->constrained()->cascadeOnDelete();
$table->morphs('taggable');
$this->morphToMany(Tag::class, 'taggable');         // taggable side
$this->morphedByMany(User::class, 'taggable');      // tag side

// ─── aggregates ───────────────────────────────────────────────────
User::withCount('tasks');
User::withSum('tasks', 'duration');
User::withAvg('tasks', 'duration');
User::withMin('tasks', 'duration');
User::withMax('tasks', 'duration');

// ─── N+1 defenses ─────────────────────────────────────────────────
Model::preventLazyLoading(! app()->isProduction());
Model::automaticallyEagerLoadRelationships();
$collection->withRelationshipAutoloading();
$user->loadMissing(['tasks', 'teams']);

// ─── morph map (always do this) ───────────────────────────────────
Relation::enforceMorphMap([
    'users' => \App\Models\User::class,
    'teams' => \App\Models\Team::class,
    'tasks' => \App\Models\Task::class,
]);
```

Study Workflow

1. Watch the episode.
2. Hit me with **"quiz me on Eloquent Episode X"** — get a coding challenge.
3. Build it from scratch (no peeking).
4. Paste back for feedback.
5. Move on when you can explain the relationship in your own words.

Red flag: if you're aggregating in PHP (`->sum('foo')` after fetching), or accessing a relation you didn't `with()`, stop and ask — should this be a `withSum`? Should I turn on auto-loading?